GitHub: https://github.com/mac2218/IA-Lab07.git

1. Clases de Tic Tac Toe

```
import math
import time
import random
from copy import deepcopy


class TicTacToeEngine:

    # Inicializa tablero 3x3 o 4x4
    def __init__(self, size=3):
        if size not in [3, 4]:
            raise ValueError("size debe ser 3 o 4")

        self.size = size
        self.board = [[" " for _ in range(size)] for _ in range(size)]
        self.nodes = 0

    # Reinicia contador de nodos
    def reset_nodes(self):
        self.nodes = 0

    # Copia del estado actual
    def clone(self):
        new_game = TicTacToeEngine(self.size)
        new_game.board = deepcopy(self.board)
        return new_game

    # Celda disponible
    def is_empty(self, row, col):
        return self.board[row][col] == " "

    # Movimientos disponibles
    def get_moves(self):
        return [
            (r, c)
            for r in range(self.size)
            for c in range(self.size)
            if self.is_empty(r, c)
        ]

    # Colocar ficha
    def make_move(self, row, col, player):
        self.board[row][col] = player

    # Deshacer movimiento
    def undo_move(self, row, col):
```

```python
        self.board[row][col] = " "

    # Tablero lleno
    def is_full(self):
        return len(self.get_moves()) == 0

    # Verifica ganador
    def is_winner(self, player):
        n = self.size

        # Filas
        for r in range(n):
            if all(self.board[r][c] == player for c in range(n)):
                return True

        # Columnas
        for c in range(n):
            if all(self.board[r][c] == player for r in range(n)):
                return True

        # Diagonal principal
        if all(self.board[i][i] == player for i in range(n)):
            return True

        # Diagonal secundaria
        if all(self.board[i][n - 1 - i] == player for i in range(n)):
            return True

        return False

    # Estado terminal
    def terminal(self):
        return (
            self.is_winner("X")
            or self.is_winner("O")
            or self.is_full()
        )

    # Función heurística
    def evaluate(self):
        if self.is_winner("X"):
            return 1000

        if self.is_winner("O"):
            return -1000

        score = 0
        n = self.size
        lines = []
```

```
        # Filas
        for r in range(n):
            lines.append([self.board[r][c] for c in range(n)])

        # Columnas
        for c in range(n):
            lines.append([self.board[r][c] for r in range(n)])

        # Diagonales
        lines.append([self.board[i][i] for i in range(n)])
        lines.append([self.board[i][n - 1 - i] for i in range(n)])

        for line in lines:
            x = line.count("X")
            o = line.count("O")

            if x > 0 and o == 0:
                score += 10 ** x

            elif o > 0 and x == 0:
                score -= 10 ** o

        return score

    # Minimax puro
    def minimax_pure(self, maximizing=True):
        self.nodes += 1

        if self.terminal():
            return self.evaluate(), None

        moves = self.get_moves()

        if maximizing:
            best_score = -float("inf")
            best_move = None

            for move in moves:
                r, c = move
                self.make_move(r, c, "X")

                score, _ = self.minimax_pure(False)

                self.undo_move(r, c)

                if score > best_score:
                    best_score = score
                    best_move = move

            return best_score, best_move
```

```
        else:
            best_score = float("inf")
            best_move = None

            for move in moves:
                r, c = move
                self.make_move(r, c, "O")

                score, _ = self.minimax_pure(True)

                self.undo_move(r, c)

                if score < best_score:
                    best_score = score
                    best_move = move

            return best_score, best_move

    # Minimax limitado
    def minimax_limit(self, depth, maximizing=True):
        self.nodes += 1

        if depth == 0 or self.terminal():
            return self.evaluate(), None

        moves = self.get_moves()

        if maximizing:
            best_score = -float("inf")
            best_move = None

            for move in moves:
                r, c = move
                self.make_move(r, c, "X")

                score, _ = self.minimax_limit(depth - 1, False)

                self.undo_move(r, c)

                if score > best_score:
                    best_score = score
                    best_move = move

            return best_score, best_move

        else:
            best_score = float("inf")
            best_move = None

            for move in moves:
                r, c = move
```

```
                self.make_move(r, c, "O")

                score, _ = self.minimax_limit(depth - 1, True)

                self.undo_move(r, c)

                if score < best_score:
                    best_score = score
                    best_move = move

            return best_score, best_move

    # Alpha-Beta pruning
    def alpha_beta(self, depth, alpha, beta, maximizing=True):
        self.nodes += 1

        if depth == 0 or self.terminal():
            return self.evaluate(), None

        moves = self.get_moves()

        if maximizing:
            best_score = -float("inf")
            best_move = None

            for move in moves:
                r, c = move
                self.make_move(r, c, "X")

                score, _ = self.alpha_beta(
                    depth - 1, alpha, beta, False
                )

                self.undo_move(r, c)

                if score > best_score:
                    best_score = score
                    best_move = move

                alpha = max(alpha, best_score)

                if beta <= alpha:
                    break

            return best_score, best_move

        else:
            best_score = float("inf")
            best_move = None

            for move in moves:
```

```
                r, c = move
                self.make_move(r, c, "O")

                score, _ = self.alpha_beta(
                    depth - 1, alpha, beta, True
                )

                self.undo_move(r, c)

                if score < best_score:
                    best_score = score
                    best_move = move

                beta = min(beta, best_score)

                if beta <= alpha:
                    break

            return best_score, best_move

    # Simulación aleatoria
    def random_playout(self, turn):
        sim = self.clone()

        while not sim.terminal():
            move = random.choice(sim.get_moves())
            sim.make_move(move[0], move[1], turn)
            turn = "O" if turn == "X" else "X"

        if sim.is_winner("X"):
            return 1
        if sim.is_winner("O"):
            return -1
        return 0

    # Monte Carlo Tree Search
    def mcts(self, iterations=500, C=math.sqrt(2), player="X"):
        self.reset_nodes()

        moves = self.get_moves()

        if len(moves) == 1:
            return moves[0]

        stats = {
            move: {"wins": 0, "visits": 0}
            for move in moves
        }

        for _ in range(iterations):
```

```
            # UCT
            parent_visits = (
                sum(stats[m]["visits"] for m in moves) + 1
            )

            best_move = None
            best_score = -float("inf")

            for move in moves:
                visits = stats[move]["visits"]

                if visits == 0:
                    score = float("inf")
                else:
                    mean_win_rate = stats[move]["wins"] / visits
                    score = mean_win_rate + C * math.sqrt(
                        math.log(parent_visits) / visits
                    )

                if score > best_score:
                    best_score = score
                    best_move = move

            sim = self.clone()
            r, c = best_move
            sim.make_move(r, c, player)

            next_turn = "O" if player == "X" else "X"
            result = sim.random_playout(next_turn)

            if player == "O":
                result *= -1

            stats[best_move]["wins"] += result
            stats[best_move]["visits"] += 1

            self.nodes += 1

        return max(
            moves,
            key=lambda m: stats[m]["visits"]
        )


class GameLoop:

    # Configuración inicial
    def __init__(
        self,
        size=3,
```

```python
        mode="H-H",
        starting_player="H",
        ia_configs=None
    ):
        self.game = TicTacToeEngine(size)
        self.size = size
        self.mode = mode
        self.starting_player = starting_player

        if ia_configs is None:
            ia_configs = {
                "IA1": {
                    "algorithm": "alpha_beta",
                    "depth": 4,
                    "N": 500,
                    "C": math.sqrt(2)
                },
                "IA2": {
                    "algorithm": "mcts",
                    "depth": 4,
                    "N": 500,
                    "C": math.sqrt(2)
                }
            }

        self.ia_configs = ia_configs

    # Mostrar tablero
    def print_board(self):
        n = self.size

        for r in range(n):
            print(" | ".join(self.game.board[r]))
            if r < n - 1:
                print("-" * (4 * n - 3))
        print()

    # Movimiento humano
    def human_move(self):
        while True:
            try:
                r = int(input("Fila: "))
                c = int(input("Columna: "))

                if (
                    0 <= r < self.size
                    and 0 <= c < self.size
                    and self.game.is_empty(r, c)
                ):
                    return (r, c)
```

```
            except:
                pass

            print("Movimiento inválido")

    # Movimiento IA
    def ai_move(self, player, config):
        data = self.ia_configs[config]
        algo = data["algorithm"]

        self.game.reset_nodes()
        start = time.time()

        if algo == "minimax":

            if self.size == 3:
                _, move = self.game.minimax_pure(
                    maximizing=(player == "X")
                )
            else:
                _, move = self.game.minimax_limit(
                    data["depth"],
                    maximizing=(player == "X")
                )

        elif algo == "alpha_beta":

            _, move = self.game.alpha_beta(
                data["depth"],
                -float("inf"),
                float("inf"),
                maximizing=(player == "X")
            )

        elif algo == "mcts":

            move = self.game.mcts(
                iterations=data["N"],
                C=data["C"],
                player=player
            )

        else:
            raise ValueError("Algoritmo inválido")

        elapsed = time.time() - start

        print("Nodos explorados:", self.game.nodes)
        print("Tiempo:", round(elapsed, 4), "seg")
```

```python
        return move

    # Ejecución del juego
    def play(self):

        # H-H
        if self.mode == "H-H":
            current = "X"

        # H-IA
        elif self.mode == "H-IA":
            current = "X"

        # IA-IA
        elif self.mode == "IA-IA":
            current = "X" if self.starting_player == "IA" else "O"

        else:
            raise ValueError("Modo inválido")

        while not self.game.terminal():

            self.print_board()

            # Humano vs Humano
            if self.mode == "H-H":
                move = self.human_move()

            # Humano vs IA
            elif self.mode == "H-IA":

                if (
                    self.starting_player == "H" and current == "X"
                ) or (
                    self.starting_player == "IA" and current == "O"
                ):
                    move = self.human_move()
                else:
                    move = self.ai_move(current, "IA1")

            # IA vs IA
            else:
                if current == "X":
                    move = self.ai_move("X", "IA1")
                else:
                    move = self.ai_move("O", "IA2")

            self.game.make_move(move[0], move[1], current)
            current = "O" if current == "X" else "X"
```

```
        self.print_board()

        if self.game.is_winner("X"):
            print("Ganó X")
        elif self.game.is_winner("O"):
            print("Ganó O")
        else:
            print("Empate")
```

2. Explosión Combinatoria

a) Tic Tac Toe 3x3

Busqueda Minimax

```
import pandas as pd
import time

resultados_minimax = []

for depth in range(1, 10):
    game = TicTacToeEngine(size=3)

    game.reset_nodes()
    inicio = time.time()

    _, move = game.minimax_limit(depth, maximizing=True)

    fin = time.time()

    resultados_minimax.append({
        "Depth": depth,
        "Nodos": game.nodes,
        "Tiempo (seg)": round(fin - inicio, 4)
    })

df_minimax = pd.DataFrame(resultados_minimax)
print(df_minimax)
```

```
   Depth   Nodos  Tiempo (seg)
0      1      10        0.0000
1      2      82        0.0021
2      3     586        0.0099
3      4    3610        0.0561
4      5   18730        0.2663
5      6   73450        1.0694
6      7  221626        2.6726
7      8  422074        5.4701
8      9  549946        7.0040
```

Busqueda Minimax con poda α-β

```
resultados_ab = []

for depth in range(1, 10):
    game = TicTacToeEngine(size=3)

    game.reset_nodes()
    inicio = time.time()

    _, move = game.alpha_beta(
        depth,
        -float("inf"),
        float("inf"),
        maximizing=True
    )

    fin = time.time()

    resultados_ab.append({
        "Depth": depth,
        "Nodos": game.nodes,
        "Tiempo (seg)": round(fin - inicio, 4)
    })

df_ab = pd.DataFrame(resultados_ab)
print(df_ab)
```

```
   Depth  Nodos  Tiempo (seg)
0      1     10        0.0000
1      2     36        0.0010
2      3    167        0.0021
3      4    635        0.0130
4      5   2805        0.0410
5      6   4632        0.0700
6      7  12614        0.1752
7      8  13451        0.1884
8      9  18297        0.2420
```

b) Tic Tac Toe 4x4

Busqueda con α-β

```
resultados_4x4 = []

for depth in range(1, 7):
    game = TicTacToeEngine(size=4)

    game.reset_nodes()
    inicio = time.time()
```

```
    _, move = game.alpha_beta(
        depth,
        -float("inf"),
        float("inf"),
        maximizing=True
    )

    fin = time.time()

    branching = game.nodes ** (1 / depth)

    resultados_4x4.append({
        "Depth": depth,
        "Nodos": game.nodes,
        "Tiempo (seg)": round(fin - inicio, 4),
        "Branching": round(branching, 4)
    })

df_4x4 = pd.DataFrame(resultados_4x4)
print(df_4x4)
```

```
   Depth  Nodos  Tiempo (seg)  Branching
0      1     17        0.0010    17.0000
1      2     47        0.0011     6.8557
2      3    313        0.0104     6.7897
3      4   3169        0.0685     7.5029
4      5  18674        0.3785     7.1490
5      6  50239        1.0538     6.0744
```

3. Duelo de Algoritmos (IA-IA)

```
import random
import math
import copy
import time

class TicTacToeEngine:
    def __init__(self, size=3):
        self.size = size
        self.board = [[' ']*size for _ in range(size)]

    def get_moves(self):
        return [(i, j) for i in range(self.size) for j in
range(self.size) if self.board[i][j] == ' ']

    def make_move(self, move, player):
        i, j = move
```

```
        self.board[i][j] = player

    def is_winner(self, player):
        for i in range(self.size):
            if all(self.board[i][j] == player for j in
range(self.size)):
                return True
            if all(self.board[j][i] == player for j in
range(self.size)):
                return True

        if all(self.board[i][i] == player for i in range(self.size)):
            return True
        if all(self.board[i][self.size-1-i] == player for i in
range(self.size)):
            return True

        return False

    def clone(self):
        new = TicTacToeEngine(self.size)
        new.board = copy.deepcopy(self.board)
        return new

    def alpha_beta(self, depth, alpha, beta, maximizing=True,
root=True):
        if self.is_winner('X'):
            return 1
        if self.is_winner('O'):
            return -1
        if depth == 0 or not self.get_moves():
            return 0

        if maximizing:
            max_eval = -math.inf
            best_move = None

            for move in self.get_moves():
                new = self.clone()
                new.make_move(move, 'X')
                eval = new.alpha_beta(depth-1, alpha, beta, False,
False)

                if eval > max_eval:
                    max_eval = eval
                    best_move = move

                alpha = max(alpha, eval)
                if beta <= alpha:
                    break
```

```
            return best_move if root else max_eval

        else:
            min_eval = math.inf

            for move in self.get_moves():
                new = self.clone()
                new.make_move(move, 'O')
                eval = new.alpha_beta(depth-1, alpha, beta, True,
False)

                min_eval = min(min_eval, eval)
                beta = min(beta, eval)

                if beta <= alpha:
                    break

            return min_eval

    def simulate_random_game(self, player):
        current = player
        state = self.clone()

        while True:
            moves = state.get_moves()
            if not moves:
                return 0

            move = random.choice(moves)
            state.make_move(move, current)

            if state.is_winner(current):
                return 1 if current == 'X' else -1

            current = 'O' if current == 'X' else 'X'

    def mcts(self, N, C):
        moves = self.get_moves()
        scores = {move: 0 for move in moves}

        for move in moves:
            for _ in range(N):
                new = self.clone()
                new.make_move(move, 'X')
                result = new.simulate_random_game('O')
                scores[move] += result

        return max(scores, key=scores.get)
```

```python
def play_game(engine, ia1_config, ia2_config):
    current_player = 'X'
    times = {'IA1': [], 'IA2': []}

    while True:
        start = time.time()

        if current_player == 'X':
            move = engine.mcts(ia1_config['N'], ia1_config['C'])
            times['IA1'].append(time.time() - start)
        else:
            move = engine.alpha_beta(
                depth=ia2_config['depth'],
                alpha=float('-inf'),
                beta=float('inf')
            )
            times['IA2'].append(time.time() - start)

        engine.make_move(move, current_player)

        if engine.is_winner(current_player):
            return current_player, times

        if not engine.get_moves():
            return 'Draw', times

        current_player = 'O' if current_player == 'X' else 'X'


def run_experiment(n_games=20):
    results = {'X': 0, 'O': 0, 'Draw': 0}
    all_times = {'IA1': [], 'IA2': []}

    ia1_config = {'N': 50, 'C': 2**0.5}
    ia2_config = {'depth': 4}

    for i in range(n_games):
        print(f"Juego {i+1}")

        engine = TicTacToeEngine(size=3)
        winner, times = play_game(engine, ia1_config, ia2_config)

        print("Ganador:", winner)

        results[winner] += 1
        all_times['IA1'] += times['IA1']
        all_times['IA2'] += times['IA2']

    avg_time_ia1 = sum(all_times['IA1']) / len(all_times['IA1'])
    avg_time_ia2 = sum(all_times['IA2']) / len(all_times['IA2'])
```

```python
    print("\n===== RESULTADOS =====")
    print(results)

    print("\nTiempo promedio por jugada:")
    print(f"IA-1 (MCTS): {avg_time_ia1:.4f} s")
    print(f"IA-2 (Minimax): {avg_time_ia2:.4f} s")


run_experiment(20)
```

```
Juego 1
Ganador: X
Juego 2
Ganador: X
Juego 3
Ganador: X
Juego 4
Ganador: X
Juego 5
Ganador: X
Juego 6
Ganador: X
Juego 7
Ganador: X
Juego 8
Ganador: X
Juego 9
Ganador: X
Juego 10
Ganador: X
Juego 11
Ganador: X
Juego 12
Ganador: X
Juego 13
Ganador: X
Juego 14
Ganador: X
Juego 15
Ganador: X
Juego 16
Ganador: X
Juego 17
Ganador: Draw
Juego 18
Ganador: X
Juego 19
Ganador: X
Juego 20
Ganador: X
```

```
===== RESULTADOS =====
{'X': 19, 'O': 0, 'Draw': 1}

Tiempo promedio por jugada:
IA-1 (MCTS): 0.0180 s
IA-2 (Minimax): 0.0042 s
```

Pregunta: ¿Cuál es más ”inteligente” en términos de ganar y cuál es más ”eficiente” en términos de tiempo por jugada?

En los experimentos realizados, la IA basada en MCTS resultó ser más "inteligente" en términos de ganar, ya que obtuvo 19 victorias de 20 partidas, superando ampliamente a Minimax con poda alpha-beta.

Por otro lado, Minimax con poda alpha-beta fue más "eficiente" en términos de tiempo por jugada, con un promedio de 0.0042 s frente a 0.0180 s de MCTS.

Conclusión:

- MCTS es más inteligente (gana más partidas)
- Minimax + alpha-beta es más eficiente (más rápido por jugada)

4. Discusion "Para pensar"

4. Restricción de tiempo (1 segundo por jugada)

Cuando la IA tiene un límite de tiempo, el algoritmo debe devolver la mejor jugada encontrada hasta el momento.

Propuestas

- **Iterative Deepening (Minimax)**
  - Ejecutar depth = 1, 2, 3...
  - Guardar la mejor jugada de la última iteración completa
  - Se puede detener en cualquier momento
- **MCTS limitado por tiempo**
  - Ejecutar simulaciones hasta que se acabe el tiempo
  - Elegir la jugada más visitada
- **Anytime Algorithms**
  - Siempre tienen una solución parcial válida
  - Mejoran con más tiempo
- **Optimización**
  - Move ordering
  - Tablas de transposición
  - Poda más agresiva

- **Paralelización**
  - Simulaciones MCTS en paralelo
  - Evaluación de ramas simultánea

Conclusión

Las mejores opciones bajo restricción de tiempo son:

- MCTS limitado por tiempo
- Minimax con iterative deepening

Ambos garantizan una respuesta en menos de 1 segundo.